Rajshahi University of Engineering & Technology

Department of Electrical and Computer Engineering

MarketNest

Real-World Database System Design and Implementation

COURSE	ECE 2216 - Database Systems Project
SUBMITTED BY	Mustakim Al Siyam | ID 2310047 | Registration 1100
SUBMITTED TO	ECE 2216 Course Instructor
SUBMISSION DATE	10 November 2026

Project scope

A relational database implementation for a multi-vendor marketplace, with transaction controls, sample data, analytical queries, and a simple administrator dashboard.

Report guide

This report presents the implemented database system, explains how its integrity rules work, and records the test plan and latest execution results. The complete executable SQL and application source are included in the submission bundle.

Section	Content
1	Introduction, problem, and objectives
2	Users, requirements, and business rules
3	Relational design and integrity constraints
4	Functional dependencies and normalization
5	MySQL implementation and sample data
6	Views, procedures, triggers, and transactions
7	SQL query development
8	Dashboard and user operations
9	Testing, limitations, and conclusion
References	Course and technical sources

Project at a glance

Measure	Implementation
Database	MySQL 8.0.16+ with InnoDB
Relations	18 tables aligned with the submitted ER model
Sample data	At least 10 rows in every table
SQL development	29 meaningful queries across six course categories
Advanced features	3 views, 3 stored procedures, 13 triggers
Application	Dashboard supports insert, search/filter, update, guarded delete, reporting, and data visualization
Testing	34 documented cases plus a runnable MySQL result recorder

MarketNest database design diagrams

The conceptual diagram records entities, relationships, cardinality, participation, and weak entities. The relational EER diagram shows the table mapping used as the starting point for this implementation.

Conceptual ER diagram

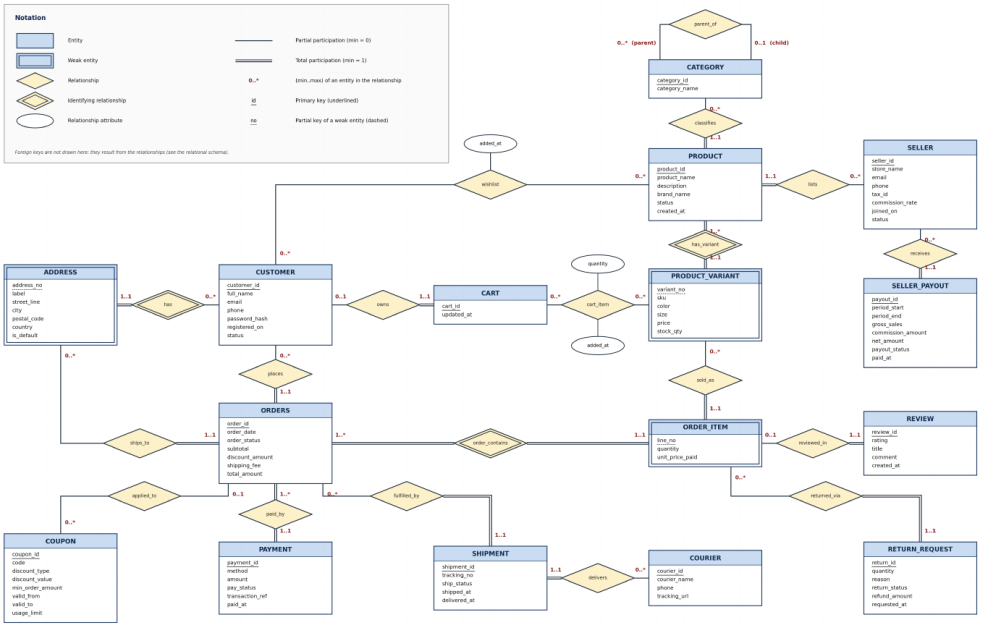


Figure 3.1: MarketNest Conceptual ER Diagram

Relational EER diagram

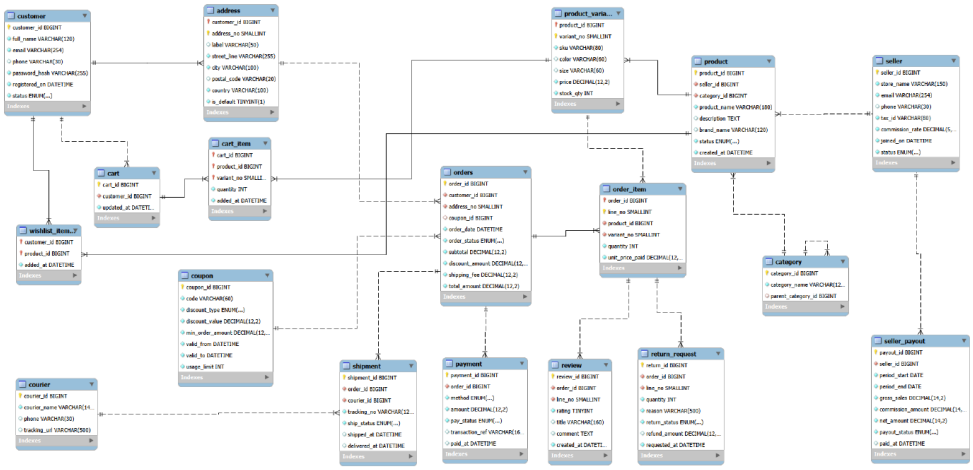


Figure 3.2: MarketNest Relational (EER) Diagram from MySQL Workbench

1 Introduction

Small and medium-sized sellers often manage products and orders across messages, spreadsheets, and separate marketplace tools. That separation can produce stale stock values, inconsistent order prices, missed delivery updates, and unclear seller settlements. MarketNest brings customer, seller, catalogue, order, payment, delivery, return, and payout records into one relational database.

The project extends the submitted MarketNest design into an implementation. It preserves the existing entities and key relationships, then adds database objects and a modest dashboard to demonstrate everyday operations.

Objectives

The system stores marketplace records in normalized tables; maintains variant-level price and stock; preserves purchase-time prices and order addresses; supports checkout, shipment, review, return, and payout workflows; and provides operational reports for sellers and administrators.

System boundary

The implementation demonstrates core database operations using synthetic data. It does not connect to a real payment gateway, execute courier bookings, authenticate marketplace users, or provide a full accounting ledger. Those features are outside the scope of the classroom database project.

2 Users, requirements, and rules

User	Main system tasks
Customer	Search products, manage a cart and wishlist, place orders, track delivery, review purchases, and request returns.
Seller	Maintain product listings and variants, adjust stock, inspect sales, and review commission and payout records.
Administrator	Manage seller and catalogue records, coupons, delivery, returns, payouts, and marketplace reports.
Courier staff	Update tracking and shipment status for assigned parcels.
Payment gateway	Return payment-attempt outcomes and transaction references to the marketplace.

Implemented functional areas

Area	Database support
Accounts and addresses	Customer and seller records; customer-scoped addresses with a composite key and one-default-address constraint.
Catalogue	Seller-owned products, hierarchical categories, and SKU-unique variants with price and quantity.
Cart and wishlist	Associative relations preserve quantity and saved time while preventing duplicate item pairs.
Orders and payments	Order totals, coupon reference, historical order-line prices, multiple payment attempts, and payment status.
Fulfilment and after-sales	Courier shipments, delivered-item reviews, seven-day returns, and refund records.
Finance and reports	Seller commissions, payout periods, low-stock, product, order, rating, and return reports.

Business rules

Each order belongs to one customer and references an address belonging to that same customer. Each order line identifies one product variant and records the unit price paid at checkout. Stock cannot be negative; a new order line is rejected when requested quantity exceeds available stock. A cancellation is allowed only before shipment and restores reserved stock. A review requires delivery. A return must be made within seven days of delivery, and non-rejected return quantities cannot exceed the quantity purchased. Seller payout totals equal gross sales minus commission, and payout date ranges for one seller cannot overlap.

3 Relational design

The database retains the 18 relations from the conceptual model. The weak-entity relationships for addresses, variants, and order lines are represented with composite primary keys where appropriate. The two many-to-many relations, cart items and wishlist items, are associative tables.

Relation	Purpose	Primary key
customers	Customer identity and account status	customer_id
addresses	Saved customer delivery addresses	customer_id, address_no
sellers	Marketplace stores and commission rate	seller_id
categories	Product taxonomy with parent category	category_id
products	Seller-owned catalogue listings	product_id
product_variants	Sellable SKU, price, and stock	product_id, variant_no
carts	One active cart per customer	cart_id
cart_items	Cart variant and requested quantity	cart_id, product_id, variant_no
wishlist_items	Saved customer-product pairs	customer_id, product_id
coupons	Discount rules and validity dates	coupon_id
orders	Customer checkout and order totals	order_id
order_items	Purchased variants and paid prices	order_id, line_no
payments	Payment attempts and transaction references	payment_id
couriers	Courier company and tracking details	courier_id
shipments	Order parcels and delivery status	shipment_id
reviews	Rating attached to a delivered order line	review_id
return_requests	Requested return units and refund	return_id
seller_payouts	Period sales and seller settlement	payout_id

The relational schema in `sql/01_schema.sql` defines every column, key, foreign key, uniqueness rule, check, and performance index. The composite foreign key from orders to addresses ensures that an order cannot use an address belonging to a different customer. A stored generated key and unique constraint allow at most one default address per customer.

Key integrity rules

Rule	Enforcement
Unique identity fields	UNIQUE constraints for email, tax identifier, SKU, coupon code, courier, tracking number, and payment reference.
Quantity and price boundaries	CHECK constraints for positive quantities, nonnegative prices and stock, valid ratings, and valid discounts.
Order arithmetic	CHECK constraint requires total = subtotal - discount + shipping.
Relationships	Foreign keys enforce customer, address, seller, catalogue, order, payment, shipment, review, return, and payout links.
Cross-row business logic	Triggers and stored procedures handle stock, order lifecycle, delivery eligibility, return window/quantity, cancellation restoration, and payout overlap.

4 Functional dependencies and normalization

The normalization example starts with an order form that stores order, customer, shipping address, coupon, seller, product, variant, and multiple purchased-line details together. The repeating group of order lines means the unnormalized relation is not in first normal form.

For the order-form example, the candidate key is (order_id, sku): one order can contain several SKUs, and one SKU can appear on many orders. The order identifier determines order date, customer, address, and coupon. The SKU determines product and variant details. The combination of order_id and SKU determines line quantity and purchase-time price.

Stage	Change	Reason
UNF	Order header plus a repeating set of product lines	An order can contain multiple variant rows.
1NF	Place one order line on each row	Every field contains one atomic value.
2NF	Split order header, order line, and variant information	Remove dependencies on only order_id or SKU when the key is composite.
3NF	Separate customer, address, coupon, seller, and category descriptions	Remove transitive dependencies from non-key attributes.
BCNF	Check that each listed determinant is a superkey	The decomposed relations meet BCNF under the stated dependencies.

Implementation keys

The physical schema uses a weak-entity key of (order_id, line_no) for order_items, consistent with the submitted ER model, and also enforces uniqueness of (order_id, product_id, variant_no). The product variant relation uses (product_id, variant_no) as its primary key and keeps SKU globally unique. Purchase-time unit_price_paid is stored on each order line, so later catalogue price updates do not alter historical orders.

5 MySQL implementation and sample data

The project uses MySQL with the InnoDB storage engine. Primary and foreign keys protect relationships, CHECK and UNIQUE constraints protect row-level rules, and indexes support frequent joins and filters. `sql/01_schema.sql` creates the database. `sql/02_seed_data.sql` inserts deterministic synthetic data. `sql/03_database_features.sql` installs views, triggers, and procedures.

Sample data coverage

Each of the 18 relations contains at least ten records to support joins and reports. Customer and seller emails use the reserved example.test domain. The sample records are fictional and contain no actual payment card details.

Relation	Rows
customers	10
addresses	15
sellers	10
categories	12
products	20
product_variants	30
carts	10
cart_items	20
wishlist_items	20
coupons	10
orders	15
order_items	20
payments	15
couriers	10
shipments	13
reviews	10
return_requests	10
seller_payouts	10

The SQL scripts must be run in this order: schema, seed data, then database features. The feature script is safe to re-run because it replaces views and drops/recreates its triggers and procedures. The seed script uses fixed IDs and is intended for a fresh database only.

6 Views, procedures, and triggers

Views

View	Purpose
v_product_catalog	Combines product, seller, category, variant, stock, and review summary for searching.
v_order_summary	Shows order totals with successful payment sums and shipment counts.
v_seller_monthly_sales	Summarizes delivered sales, estimated commission, and net sales by seller and month.

Stored procedures

Procedure	Operation
sp_reserve_order_line	Locks a variant, checks available stock, inserts a historical order line, and decreases stock inside the caller's transaction.
sp_cancel_order	Runs an atomic cancellation for a pending or confirmed order; updates payment state and invokes the stock-restoration trigger.
sp_generate_seller_payout	Calculates delivered line revenue for a seller and inclusive date range, derives commission, inserts a payout, and returns the new payout ID.

Triggers

Trigger	Purpose
ai_categories_no_self_parent	Rejects self-parenting categories after the generated category ID is assigned.
bu_categories_no_self_parent	Rejects updates that make a category its own parent.
bi_orders_validate_coupon	Checks coupon validity, minimum spend, discount amount, and usage limit on new orders.
bi_order_item_check_stock	Locks and checks the selected variant before a line is inserted.
bu_orders_validate_transition	Rejects invalid lifecycle changes and cancellation after shipment.
au_orders_restore_cancelled_stock	Restores each line's stock for an eligible cancellation.
bi_reviews_require_delivery	Requires a delivered order before a review is inserted.
bi_returns_validate_request	Checks delivery, the seven-day window, and cumulative return quantity.
bu_returns_validate_quantity	Preserves the return quantity limit when a request is updated.
bi_payouts_no_overlap	Rejects overlapping seller payout dates on insert.
bu_payouts_no_overlap	Rejects overlapping seller payout dates on update.

Transactions

Checkout calls the stock-reservation procedure inside an application-managed transaction, then records payment and clears the cart before commit. If any line fails the stock check, the application rolls the complete checkout back. The cancellation procedure uses a transaction and row lock; its order-status update and stock-restoration trigger execute together. Product and first-variant creation are also committed together.

7 SQL query development

The query set contains 29 business questions and uses basic filtering, joins, grouping, HAVING, subqueries, arithmetic and logical operators, common table expressions, window functions, and ranking. The full executable statements are labeled Q01-Q29 in `sql/04_queries.sql`.

Category	Count	Examples of concepts
Basic queries	4	Selection, range filters, stock checks
JOIN queries	7	INNER JOIN, LEFT JOIN, multi-table joins
Aggregates and HAVING	5	SUM, COUNT, AVG, GROUP BY, HAVING
Subqueries	4	Scalar, derived-table, EXISTS, correlated EXISTS
Arithmetic and logical	3	Computed commission, IN, AND conditions
Complex analytics	6	Ranking, CTE, LAG, CUME_DIST, return rate

ID	Problem answered
Q01	Active catalogue with available stock
Q02	Orders during a selected date range
Q03	Variants inside a selected price range
Q04	Low-stock variants
Q05	Customer order history and address
Q06	Product, seller, category, and stock
Q07	Order-line detail with purchase-time price
Q08	Tracking and courier contact
Q09	Payment attempts and order totals
Q10	Return request and purchased item
Q11	Products with no reviews
Q12	Delivered sales by seller with HAVING
Q13	Categories with multiple listings
Q14	Monthly orders and average basket
Q15	Product ratings with threshold
Q16	Return volume by status
Q17	Products above average price
Q18	Customers above average delivered spend
Q19	Sellers with low-stock variants
Q20	Products with delivered sales
Q21	Line revenue and commission
Q22	Pending or failed payment orders
Q23	Active listings with low stock
Q24	Top products by delivered quantity
Q25	Monthly seller revenue ranking
Q26	Monthly revenue change using LAG
Q27	Coupon usage and order value
Q28	High-spend percentile using CUME_DIST
Q29	Return rate by product

8 Dashboard application

The MarketNest operations dashboard connects to the same MySQL database. It supports record creation, keyword and category/seller/price/rating search, profile and listing updates, guarded deletion, cart and wishlist management, coupon checkout, inventory and price changes, order confirmation and fulfilment, returns and refunds, reviews, payment outcomes, payouts, and reports with charts. A manual Refresh data control reloads tables after saved changes. Records linked to order history are retained.

Configuration and privacy

Connection settings are read from a local `.env` file. The example file uses placeholders only. The app has no production authentication; the optional role template demonstrates how an administrator can grant limited database privileges. The app never stores card data or connects to an external payment service.

Demonstration steps

Import the SQL files, configure MySQL, and install `requirements.txt`. Launch with `streamlit run app/app.py --server.address 127.0.0.1`. Demonstrate seller approval, color/size variants, customer checkout, order updates, returns, and report charts.

9 Testing and results

The package includes 34 test cases and a Python runner that exercises constraints and database objects against a live MySQL instance. It covers valid data, invalid values, duplicate data, foreign-key failures, boundary values, transaction rollback, trigger behavior, views, procedures, query output, and rejected credentials.

Test execution status

All 34 documented test cases passed against a fresh MySQL 8.0.46 database during validation. The included `tests/test_results.csv` records each observed result. Run the test runner again after importing the scripts into your own MySQL server.

Test area	Cases	Expected outcome	Current status
Sample volume and schema	TC01	18 tables; >=10 rows each	PASS
Constraints and keys	TC02-TC10, TC12, TC19, TC23	Invalid stock, duplicate IDs, invalid references and totals are rejected	PASS
Triggers and order rules	TC11, TC13-TC14, TC21-TC22, TC25-TC28, TC30-TC34	Delivery, stock, return lifecycle, payout, coupon, and category rules are enforced	PASS
Database objects and workflows	TC15-TC17, TC24, TC29	Views, procedures, cancellation, and catalog queries work	PASS
Transactions and access	TC18, TC20	Rollback restores state; invalid credentials are refused	PASS

Limitations and future work

The current demonstration uses synthetic records and a single database account configured locally. It does not integrate an external gateway, calculate carrier charges, provide multi-warehouse inventory, authenticate dashboard users, or implement a bank settlement/refund ledger. Future work could add role-based application login, a payment gateway sandbox, seller-specific access controls, stock reservation expiry, audit logs, and automated backups.

Conclusion

MarketNest implements the submitted 18-relation marketplace design with enforced constraints, sample data, reporting queries, database workflows, and an operations dashboard. Its test runner and query set support a clear demonstration in MySQL Workbench and the app.

References

[1] ECE 2216, Real-World Database System Design and Implementation, course project brief, 2026.

[2] Mustakim Al Siyam, MarketNest: Design of a Multi-Vendor E-Commerce Database System, ECE 2215 assignment report, RUET, 2026.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[4] MySQL 8.4 Reference Manual, CHECK Constraints, CREATE PROCEDURE, and Trigger Syntax, https://dev.mysql.com/doc/refman/8.4/en/.